Case Study : Smart Banking Fraud Detection System using Apache Spark

A national banking organization processes millions of financial transactions daily through ATMs, internet banking, mobile banking, and POS terminals. The bank wants to develop a **Fraud Detection and Customer Analytics Platform** using Apache Spark to identify fraudulent transactions, analyze customer behavior, and generate predictive insights. The final application should be deployed using modern DevOps and cloud-native technologies.

As a Big Data Engineer, design, implement, and deploy the solution using Apache Spark and PySpark.

Q1. Spark Initialization and Data Loading (2 Marks)

Initialize **Spark Session** and **Spark Context**. Load banking transaction datasets and display the schema and sample records.

Q2. RDD Implementation (3 Marks)

Create **RDDs** from transaction datasets and perform transformation and action operations to analyze customer transactions.

Q3. Key-Value Operations and Persistence (2 Marks)

Implement **key-value pair operations**, perform **shuffle operations**, and demonstrate **RDD persistence** for transaction processing.

Q4. Spark DataFrame Operations (3 Marks)

Convert transaction datasets into **Spark DataFrames** and perform selection, filtering, grouping, joins, and aggregation operations.

Q5. Exploratory Data Analysis and Spark SQL (5 Marks)

Perform EDA and execute Spark SQL queries to:

- Identify high-value transactions.
- Analyze customer transaction patterns.
- Determine branch-wise transaction volume.
- Identify suspicious transaction patterns.
- Generate monthly transaction trends.

Q6. ETL Pipeline Development (3 Marks)

Design and implement an ETL pipeline to extract transaction data from multiple banking systems, transform and validate the data, and load it into a centralized data repository.

Q7. Machine Learning/Deep Learning Implementation (2 Marks)

Implement a Machine Learning/Deep Learning model to predict fraudulent transactions or customer churn and evaluate its performance.

Additional Implementation Instructions

A. Version Control using Git and GitHub

- Create a **GitHub repository** for the project.
- Upload all source code, datasets (or dataset links), notebooks, and documentation.
- Maintain proper folder structure and commit history.
- Include a **README.md** file containing project description, installation steps, execution procedure, and output screenshots.

B. Containerization using Docker

- Create a **Dockerfile** for the Spark application.
- Build and test the Docker image.

C. Deployment using Kubernetes

- Create Kubernetes deployment and service YAML files.
- Deploy the application on a Kubernetes cluster (Minikube/Kubernetes/OpenShift).
- Verify successful deployment.

D. DevOps Pipeline Implementation

- Configure a basic CI/CD pipeline using GitHub Actions/Jenkins/GitLab CI.
- Automate build, testing, Docker image creation, and deployment.

E. Project Submission Requirements

Submit:

- Source code
- GitHub repository link
- Dockerfile
- Kubernetes YAML files
- CI/CD pipeline configuration
- Execution screenshots
- EDA and Spark SQL outputs
- Model evaluation report
- Final project documentation